一、问题分析

实验目的： 本实验旨在实现一个能够直接计算中缀算术表达式的程序，而不通过转换为后缀或前缀表达式。该程序将处理包含加法（+）、减法（-）、乘法（*）、除法（/）以及括号（()）的表达式，并确保所有操作数均为正数且类型为 double。

输入要求： 程序接收多行输入，每行代表一个以等号"="结尾的算术表达式。当遇到单独一行等号时，表示输入结束。

输出要求： 对于每个输入的表达式，程序应输出其运算结果，保留两位小数。

二、总体设计

为了实现上述功能，我们使用了两个栈结构：一个用于存储数字（nums），另一个用于存储运算符（operate）。算法的核心在于如何正确地处理运算符的优先级和括号逻辑，确保按照正确的数学规则执行运算。

三、详细设计

1. **函数定义：**
   - `isNum(char c)`：判断字符是否为数字或小数点。
   - `isOp(char c)`：判断字符是否为四则运算符。
   - `level(char c)`：返回运算符的优先级。
   - `cal(char op, double a, double b)`：根据给定的操作符计算两个数的结果。
   - `calTop()`：从栈顶取出运算符和相应的两个操作数进行计算，并将结果压入数字栈。
   - `calculate(string &s)`：解析并计算传入的字符串形式的表达式。
   - `main()`：主函数，负责读取用户输入并调用 `calculate` 函数。
2. **核心逻辑：**
   - 在 `calculate` 函数中，遍历输入字符串中的每一个字符，根据字符性质（数字、运算符、括号）采取不同的处理方式。
   - 当遇到运算符时，会检查当前运算符与栈顶运算符的优先级关系，必要时先计算栈内较高优先级的运算。
   - 括号的处理遵循标准数学规则，即在遇到右括号时，持续计算直到遇到对应的左括号。
   - 最终，所有剩余的运算都将在最后被计算完毕，栈底元素即为最终结果。

四、调试分析

在开发过程中，主要遇到了以下问题：

- 处理连续数字（如多位整数或浮点数）的问题。解决方法是引入了一个临时字符串 `tmpNum` 来累积完整的数字字符串，然后使用 `stod` 函数将其转换为 `double` 类型再压入栈中。
- 运算符优先级的处理。通过 `level` 函数定义了不同运算符的优先级，并在遇到新运算符时比较其与栈顶运算符的优先级，以决定是否立即计算。
- 括号匹配。确保了遇到右括号时能正确找到对应的左括号，并在此期间完成所有内部运算。

五、用户手册

使用说明：

1. 启动程序后，用户可以逐行输入中缀算术表达式，每个表达式以等号“=”结尾。
2. 程序将依次输出每个表达式的计算结果，每个结果占一行，保留两位小数。
3. 输入单独的一行等号“=”以终止程序运行。

注意事项：

1. 所有参与运算的数值必须为正数。
2. 表达式只能包含基本的四则运算符（+、-、*、/）和括号（()），并且必须格式正确。
3. 避免输入非法字符或不完整的表达式，以免造成程序错误。

六、测试结果

测试样例：

- 输入：

```
5*(1+5)*2=
5*1+5*2=
1+2/3*(4-5)=
2+2=
```

```
20*(4.5-3)=
=
```

- 输出：

```
60.00
15.00
0.33
4.00
30.00
```

程序成功实现了预期的功能，能够正确解析和计算中缀算术表达式，满足了题目要求。

附录—程序、

```
// main.cpp 文件

#include <iostream>
#include <string>
#include <vector>
#include <list>
#include <set>
#include <map>
#include <queue>
#include <algorithm>
#include <stack>
#include <iomanip>
using namespace std;


stack<double> nums;
stack<char> operate;

bool isNum(char c){
	if(c>='0'&&c<='9'||c=='.') return true;
	return false;
}

bool isOp(char c){
	return c=='+'||c=='-'||c=='*'||c=='/';
}

int level(char c){
	if(c=='+'||c=='-') return 1;
	if(c=='*'||c=='/') return 2;
	return 0;
}

double cal(char op,double a,double b){
	switch (op) {
		case '+': return a+b;
		case '-': return a-b;
		case '*': return a*b;
		case '/': if (b == 0) {
				cerr << "Error: Division by zero" << endl;
```

```
                exit(1);
            }
            return a / b;
        default: cerr << "Error: Invalid operator" << endl;
            exit(1);
    }
}

void calTop(){
    char topOp=operate.top();
    double numb = nums.top();   nums.pop();
    double numa = nums.top();   nums.pop();
    double newNum = cal(topOp, numa, numb);
    nums.push(newNum);
    operate.pop();
}

double calculate(string &s){
    string tmpNum;
    for(auto &subC:s){
        if(subC=='=') {
            if(!tmpNum.empty()){
                nums.push(stod(tmpNum));
                tmpNum.clear();
            }
            break;
        }
        if(subC==' ')
            continue;
        //如果是数字就加到字符串里
        if(isNum(subC)){
            tmpNum+= subC;
            continue;
        }
         //如果是操作符
        if(isOp(subC)){
            //将操作符前一个数字解析入栈
            if(!tmpNum.empty()){
                nums.push(stod(tmpNum));
                tmpNum.clear();
            }
            //如果操作符栈空，直接入栈
            if(operate.empty()) {
                operate.push(subC);
```

```
                continue;
            }
            char topOp=operate.top();
            //如果操作符等级不高于栈顶，则计算再入栈新操作符
            if(level(topOp)>= level((subC))){
                calTop();
            }
            //否则就入栈新操作符
            operate.push(subC);
            continue;
        }
        //如果是括号
        if(subC==')'){
            //将操作符前一个数字解析入栈
            if(!tmpNum.empty()){
                nums.push(stod(tmpNum));
                tmpNum.clear();
            }

            char topOp=operate.top();
            while(topOp!='(') {
                calTop();
                topOp=operate.top();
            }
            operate.pop();
            continue;
         }
          operate.push(subC);
    }

    while(!operate.empty())
        calTop();

    return nums.top();
}

int main()
{
    string input;
    cout<<fixed<<setprecision(2);
    while(true){
        cin>>input;
        if(input=="=")
            break;
```

```
            cout<<calculate(input)<<endl;
        }

        return 0;
}
```